

Le rôle de l'analytics engineer

À mi-chemin entre le Data Analyst et le Data Engineer, l'Analytics Engineer est un profil de plus en plus recherché dans les entreprises modernes.

Contrairement au Data Analyst, dont la principale mission est l'analyse des données, l'Analytic Engineer met l'accent sur la modélisation des données. Concrètement, ce professionnel est responsable de la transformation, des tests, du déploiement et de la documentation des données qu'il gère.

Au quotidien, les missions d'un Analytics Engineer concernent :

- la conception et l'optimisation des pipelines de données pour l'acquisition, la transformation et le chargement (ETL/ELT) ;
- l'élaboration et la gestion de modèles de données pour simplifier leur analyse ;
- la surveillance et l'amélioration des performances des systèmes de traitement des données ;
- la mise en place de pratiques de gouvernance des données pour garantir leur qualité et leur sécurité.

L'analytics engineer joue un rôle clé dans une démarche de self-service analytics en préparant et structurant les données de manière à les rendre accessibles et compréhensibles pour les utilisateurs métiers.

Il s'assure que les données sont propres, organisées, bien documentées et disponibles via des outils de visualisation ou de reporting faciles à utiliser.

Ainsi, il crée une infrastructure qui permet aux utilisateurs non techniques d'explorer et d'analyser les données en toute autonomie, tout en garantissant la qualité et la gouvernance des données.

L'Analytics Engineer joue un rôle clé au sein d'une équipe data. Afin de s'assurer que les données soient prêtes pour l'analyse et la visualisation, il conçoit, développe et maintient des pipelines de données performants. L'Analytic Engineer travaille en étroite collaboration avec les membres de son équipe pour comprendre les besoins métier et les exigences en matière de données, dans le but de proposer des solutions adaptées.

Rappel SQL

Une formation SQL est disponible dans l'espace formation Quadratic.

Premières requêtes

Afficher ses données

Les mots clés de base pour afficher les données sont **SELECT** et **FROM**. À la suite de **SELECT** on listera les colonnes que l'on souhaite afficher et à la suite de **FROM**

```
SELECT
colonne1, colonne2
FROM table1 ``
```

Filtrer

Pour filtrer les données requêtées, on utilisera le mot clé `WHERE`

```
SELECT colonne1, colonne2 FROM table1 WHERE colonne1 > 50 AND
colonne2 LIKE 'quad%' “
```

Ordonner

Le mot clé `ORDER BY` est utilisé pour ordonner les données.

```
SELECT
colonne1, colonne2
FROM table1
WHERE
colonne1 > 50
AND colonne2 LIKE 'quad%'
ORDER BY colonne1 DESC ``
```

Agréger

L'agrégation de données est le fait de regrouper les lignes d'une table selon une combinaison

```
SELECT SUM(colonne1) as total_colonne1, colonne2 FROM table1 WHERE
colonne1 > 50 AND colonne2 LIKE 'quad%' GROUP BY colonne2 ORDER BY
colonne1 DESC “
```

Créer un schéma CREATE TABLE

Nommer clairement : utiliser des noms descriptifs et cohérents pour les tables et les colonnes. Définir des types de données appropriés : choisir le type de données le plus adapté pour chaque colonne (par exemple, `INT` pour les numéros, `DATE` pour les dates). Utiliser des clés primaires : chaque table doit avoir une clé primaire pour identifier de manière unique chaque ligne. Utiliser des clés étrangères pour les relations : définir des relations entre les tables via des clés étrangères pour maintenir l'intégrité référentielle. Prévoir des contraintes pour garantir la qualité des données : comme les contraintes d'unicité, de vérification, etc. SQL - Le data definition language (DDL).

```
CREATE TABLE utilisateur (
id INT PRIMARY KEY NOT NULL,      nom VARCHAR(100),      prenom VARCHAR(100),      email VARCHAR(100)
```

Altérer ses données INSERT, UPDATE, DELETE

Il est possible d'altérer les données d'une table avec les mots clés INSERT, UPDATE, DELETE.

- INSERT permet d'ajouter des enregistrements à une table.

```
INSERT INTO table VALUES ('valeur 1', 'valeur 2', ...)
```

- UPDATE permet de changer la structure de la table comme ajouter une colonne.

```
UPDATE table
SET nom_colonne_1 = 'nouvelle valeur'
WHERE condition ``
```

- `DELETE` permet de supprimer des enregistrements à une table.

```
DELETE FROM table WHERE condition ``
```

Les Jointures

INNER JOIN et LEFT JOIN

Les deux types de jointures les plus utilisés sont INNER JOIN et LEFT JOIN :

- INNER JOIN permet d'afficher les lignes en commun entre la table de gauche et la table de droite.

```
SELECT
t1.colonne1, t2.colonne3, t1.colonne2
FROM table1 t1
INNER JOIN table2 t2
ON t1.primarykeycol = t2.secondarykeycol ``
```

- LEFT JOIN permet d'afficher toutes les lignes de la table de gauche, qu'elles aient ou non

```
SELECT t1.colonne1, t2.colonne3, t1.colonne2 FROM table1 t1 INNER JOIN
table2 t2 ON t1.primarykeycol = t2.secondarykeycol ``
```

RIGHT JOIN et FULL JOIN

- RIGHT JOIN permet d'afficher toutes les lignes de la table de droite, qu'elles aient ou non une correspondance avec la table de gauche.

Un RIGHT JOIN pourra toujours s'écrire avec un LEFT JOIN (table A LEFT JOIN table B équivaut à table B RIGHT JOIN table A).

Tout dépend de l'ordre dans lequel on souhaite écrire les tables dans la requête !

- FULL JOIN permet d'afficher toutes les lignes des tables de gauche et de droite.

Ce type de jointure est moins utilisé, car il rallonge le temps de requête : elle affiche une table avec plus de colonnes (2 colonnes ID) et potentiellement plus de lignes.

Se repérer parmi les cloud providers, optimisation et bonnes pratiques SQL pour travailler en collaboration.

Du “On premise” vers le cloud

Avant

Les entreprises achetaient et géraient leurs propres serveurs physiques. Elles s’occupaient de faire évoluer leur matériel en achetant de nouveaux composants pour ajouter de la mémoire et/ou de la puissance. Enfin, elles faisaient la maintenance de leur matériel, mais aussi les sauvegardes et étaient responsables de la sécurité des données.

Maintenant

Les entreprises se tournent de plus en plus vers des solutions cloud. Il existe trois types de services cloud :

- Les services IaaS (Infrastructure as a Service) : Ils permettent d’accéder à distance à des composants d’une structure informatique gérés par le cloud providers. Il n’y a donc pas besoin d’acheter, ni de configurer et il n’y a pas de maintenance à effectuer sur ces composants pour pouvoir bénéficier de ressources de calcul, de stockage etc ...
- Les services PaaS (Plateforme as a Service) : Ils s’appuient sur les services IaaS pour fournir des outils et services nécessaires pour créer et déployer des applications. Ce sont par exemple les systèmes d’exploitation et les middlewares.
- Les services SaaS (Software as a Service) : Ce sont des applications prêtes à l’emploi accessibles à distance et dont on ne se soucie pas du développement et de déploiement comme Google Gmail, Docs ...

Les trois grands cloud providers

Un cloud provider est une entreprise qui fournit des ressources de calcul évolutive et accessible via le web. Les trois principaux cloud providers sont :

- Google
- Amazon
- Microsoft

Cloud Provider | Data lake | Data warehouse |

Google | Google Cloud Storage | BigQuery |
Amazon | Amazon S3 | Redshift |
Microsoft | OneLake | Azure SQL server |

Chaque fournisseur a ses points forts et ses inconvénients. Les critères de choix des entreprises se font sur :

- la performance,
- le coût,
- la flexibilité.

Les grandes entreprises bénéficient en général de contrats personnalisés négociés avec les fournisseurs.

Distinguer Data Lake du Data Warehouse

Historiquement, l'ensemble des données d'une entreprise était contenu dans une base de données relationnelle (année 80). Ces bases de données reposaient sur un langage simple à utiliser (SQL) et permettaient des opérations CRUD (Create, Read, Update, Delete) qui respectaient les propriétés ACID (Atomicity, Consistency, Isolation, Durability). On utilisait ces bases de données relationnelles pour créer des rapports quotidiens à des fins opérationnelles.

Avec le temps, un besoin de réaliser des rapports analytiques a émergé. Ce besoin a nécessité d'avoir des données historiques pour projeter des tendances. Les Data Warehouses sont nées avec ce besoin. Ce sont de vastes bases de données historiques organisées par thème et sujets métiers. Ces bases de données ont gardé les capacités OLTP des bases de données relationnelles, mais apportent des fonctionnalités supplémentaires comme le processus ETL et des outils de traitement OLAP.

Bien que les data warehouse peuvent stocker des données non structurées, ils s'avèrent peu agiles devant les demandes métiers de rajout de nouvelles sources et les changements de structures de sources. C'est à partir de cet instant que sont nés les Data Lakes qui se sont avérés plus agiles que les Data Warehouse. En effet, en plus de pouvoir stocker de la donnée brute structurée ou non, les changements de structure de ces données sont rapidement pris en charge. On passe d'un schéma on write (créer un schéma pour la donnée avant de l'écrire sur la base de données) à un schéma on read (on crée le schéma d'écriture seulement en lisant la donnée).

Pour requêter un data lake on utilise un langage NoSQL.

Data lake

Un data lake est un entrepôt de données où vous pouvez stocker tous vos types de données – qu'elles soient structurées (comme des tableaux de chiffres), semi-structurées (comme des fichiers XML ou JSON), ou non structurées (comme des vidéos, des images, des fichiers audio)

Data warehouse

Un data warehouse est un grand entrepôt de données centralisées. Il est conçu pour stocker des données structurées provenant de diverses sources et les rendre faciles à analyser pour obtenir des informations utiles.

Tableau comparatif

Caractéristiques	Data lake	Data warehouse
Type de données	Structurées, semi-structurées, non structurées	Principalement structurées
Organisation	Données brutes et non organisées	Données organisées et préstructurées
Stockage	Stockage à faible cout pour grandes quantités	Stockage plus coûteux en raison de l'optimisation
Scalabilité	Très évolutif et flexible	Évolutif, mais à un coût plus élevé
Coût	Moins cher à stocker des données brutes	Plus cher en raison de l'optimisation et de la structure

Pourquoi optimiser ces requêtes

- Réduction des coûts : malgré la baisse du coût du stockage, il peut être intéressant d'optimiser ces requêtes pour qu'elles consomment moins de ressources afin de diminuer les coûts d'exploitation.
- Amélioration des performances : les dashboards et les rapports se chargent plus rapidement, ce qui offre une meilleure expérience à l'utilisateur.
- Gain de temps : les équipes peuvent accéder aux données plus rapidement, ce qui accélère le processus de prise de décision.

Partitionnement

Le partitionnement est une méthode d'indexation propre à BigQuery, les autres bases de données et les cloud providers proposent des fonctions similaires. Le partitionnement divise une table en segments plus petits, appelés partitions. Chaque partition correspond généralement à une colonne, souvent des dates, car elles répartissent les données de façon uniforme et sont fréquemment utilisées dans les filtres des requêtes. Le partitionnement permet à BigQuery de ne scanner que les partitions concernées, améliorant ainsi les performances.

Clustering

Le clustering organise physiquement les données à l'intérieur de la table en fonction des colonnes choisies. Par exemple, en clusterisant par pays, les données sont triées par pays, ce qui permet un accès plus rapide aux lignes filtrées par cette colonne. Le clustering est souvent utilisé conjointement avec le partitionnement

Bonnes pratiques

Naming

Dans l'idéal, il faut être descriptif dans les noms des colonnes, des cte et des alias de tables même si le nom est long : `ceci_est_un_exemple_de_nom`.

Utilisation des CTE

Les CTEs améliorent la lisibilité des requêtes complexes en les décomposant en étapes logiques.

Utilisation d'une indentation cohérente et homogène

On utilisera par exemple l'indentation sur les différentes clauses SELECT, FROM, WHERE, GROUP BY, etc.

Ajouter des espaces entre les opérateurs

Dans les conditions, le fait d'espacer les opérateurs (comme `=`, `<>`, `>`, `<`, etc.) améliore la lisibilité.

Ne pas faire de group by imprécis

Dans Bigquery, il est possible d'utiliser `GROUP BY ALL` ou `GROUP BY 1,2,3`.

Par exemple,

```
SELECT
categories, zone, COUNT(products)
```

```
FROM table
GROUP BY 1,2 ````
```

Dans cet exemple, on arrive facilement à identifier les colonnes qui agrègent nos données. (

Utiliser un maximum les linters

Bouton "Format" dans Bigquery

Dans la console google cloud, il faut se rendre sur la page Bigquery. En cliquant sur "Saisi

Azure SQL

Dans SQL Server Management Studio, il est possible d'installer un plug-in gratuit pour forma

Redshift

Il n'existe pas de client SQL développé par Amazon pour leurs bases de données Amazon Redsh

SQLFluff en local

SQLFluff est un linter open source, multi dialectes et configurable. C'est un package python.
La documentation se trouve à cette [adresse](https://sqlfluff.com/).

Data Build Tool : l'outil indispensable de l'AE

1. Présentation de dbt

dbt est un outil de modélisation de données qui incarne parfaitement l'approche analytics en

Il est massivement adopté par les entreprises, comme en témoigne sa fréquence de citation da

2. Centralisation et fiabilité

dbt permet de centraliser tous les modèles SQL au même endroit, organisés dans des dossiers

Cela améliore la fiabilité des données : plus de requêtes dispersées, plus de doublons, tout

3. Travail collaboratif

Tous les analystes accèdent au même référentiel SQL, ce qui facilite le travail collaborati

4. Accessibilité pour les profils SQL

dbt est conçu pour être utilisé par des profils non-développeurs, comme les data analysts qu

Cela rend ces profils autonomes pour transformer les données, en complément de leur capacité

5. Profil de l'analytics engineer

Ces outils permettent l'émergence de nouveaux profils : les analytics engineers ou data anal

Installation

Prérequis:

1. [Windows Subsystem for Linux](https://learn.microsoft.com/fr-fr/windows/wsl/install) -->
2. [UV](https://github.com/astral-sh/uv)
3. Avoir un compte d'essai gratuit Google Cloud [ici](https://cloud.google.com/bigquery?utm_

Dbt Core avec dagster

dbt Core est la version open source à installer et utiliser en local. Dagster est un data or

Dans le terminal Ubuntu, on crée notre dossier de travail avec ``uv init nom_du_projet`` et on

1. ``uv add dagster-dbt dbt-bigquery`` : On installe dagster avec dbt et son adaptateur pour Big
2. ``uv run dbt init`` : On crée notre dossier de travail dbt avec ces sous-dossiers.

- a. On donne un nom à notre projet dbt b. On sélectionne l'adaptateur que l'on va utiliser. B
- c. On sélectionne une méthode de connexion à bigquery. Je vous conseille la méthode service

- Se rendre à ce [lien](https://console.cloud.google.com/iam-admin/serviceaccounts?hl=fr&in
- Cliquer sur "Créer un compte de service" et lui donner un nom.
- À la seconde étape de création du compte de service, on donnera les rôles "Éditeur de donn
- La troisième étape est facultative, on pourra donc cliquer sur "OK".

Une fois le compte de service créé, on cliquera sur ce compte de service et on générera une

- d. On donne le nombre de coeurs CPU que DBT pourra utiliser.
- e. On donne le nombre du projet bigquery et le nom du dataset bigquery.
- f. On sélectionne la même zone géographique US ou EU pour dbt, notre projet bigquery et notr

Dbt Cloud

Pour utiliser dbtCloud, la version Saas avec une interface web de DBT, il faudra créer un co
La configuration d'un projet dbtcloud passe par les étapes ci-dessous :

1. Se connecter à une plateforme a. Cliquer sur "add new connection" b. Choisir la plateforme
- c. Charger la clé JSON du compte de service que l'on a créée précédemment pour dbt core (éta
- d. Terminer en cliquant sur save.

De l'aide est disponible, à ce [lien](https://docs.getdbt.com/docs/cloud/about-cloud-setup)

2. Configurer l'accès à Github, Gitlab ou notre propre repository git.
- Sur la page proposant de configurer un repository git, on choisira l'option "Managed" pour c

Pour finir, on choisira un nom pour notre projet dbt et il faudra cliquer sur bouton "Create

Dbt et la modélisation

Un projet dbt est constitué d'un ensemble de dossiers :

- models : Ce dossier contiendra les requêtes SQL et des fichiers YAML qui mettront en place
- snapshots : Ce dossier contiendra les fichiers de configurations des snapshots de tables.
- seeds : Ce dossier contiendra des fichiers csv qui pourront être chargés dans la plateforme
- tests : Ce dossier contiendra des tests personnalisables sur nos models.
- analysis : Ce dossier contiendra des requêtes SQL d'analyse supplémentaires.
- macros : Ce dossier contiendra des fichiers YAML de création de macros jinja. Ce code pour

Le fichier `dbt_project.yml` est le fichier principal de configuration du projet dbt. Il contient des informations sur la matérialisation de nos modèles en vue ou en table, sur le type de données, etc.

Un modèle est un fichier SQL contenant une requête améliorée avec du code jinja. Ce code jinja permet de générer la table issue du modèle `"my_first_model.sql"` on utilisera la commande ``dbt run``. À la fin de cette opération, on trouvera dans notre dataset BigQuery une nouvelle table portant le nom `"my_first_model"`.

Avant de créer son premier modèle, on va créer nos tables sources et les identifier dans un fichier `source.yml`.

- Création des tables dans bigquery

Vous trouverez le fichier `Chinook_BigQuery.sql` qui va créer les tables chinook et insérer des données. Ces tables sont décrites dans la formation `quadratic SQL`.

- Création du fichier `source.yml` dans le dossier "model" pour identifier nos sources.

Identifier les sources dans un fichier yml permet d'utiliser la fonction jinja ``{{source(source_name)}}`.

Cette référence évite par exemple d'écrire dans notre modèle ``FROM db_name.dataset.table``. De cette manière, si le dataset change, il suffit de modifier le fichier `source.yml`.

Dans l'exemple ci-dessous, notre dataset source "chinook" sera référencé par le nom `"chinook"`.
version: 2

```
sources: - name: chini_db schema: chinook tables: - name: Album description: Une table qui contient les informations concernant les albums columns: - name: AlbumId description: Identifiant unique d'un album tests: - unique - not_null - name: Artist - name: Customer
```

- Notre premier modèle

Pour faciliter la lisibilité de notre projet dbt, il est nécessaire de le structurer selon trois couches :

Staging : Les modèles de staging sont la première couche de transformation dans un projet DBT. Ils consistent en des transformations légères de vos sources de données brutes pour les préparer à des transformations plus complexes. Les modèles de staging traitent généralement une seule source à la fois et normalisent les noms des colonnes, suppriment les lignes en double, convertissent les types de données, etc.

Intermediate : Les modèles intermédiaires sont la deuxième couche de transformation. Ils effectuent des transformations plus complexes et peuvent incorporer plusieurs sources de données. Le but de ces modèles est de préparer les données pour les modèles mart.

Mart : Les modèles mart (ou data marts) sont la couche finale de transformation. Ils créent des vues finales des données qui sont optimisées pour l'analyse et la visualisation. Ces modèles consolident les informations provenant de diverses sources.

sources et les présentent dans un format facilement compréhensible et utilisable.

Nous allons donc créer dans dossier “models” un premier sous-dossier “staging” qui contiendra notre premier model nommé ‘_stg__albums.SQL’

Notre code ...

```
SELECT
AlbumId,
Title,
ArtistId,
INITCAP(Title) AS _cleaned_title,
CHAR_LENGTH(Title) AS _nb_char_in_title
FROM
{{source('chin_db', 'Album')}} ````
```

sera compilé par dbt ...

```
SELECT AlbumId, Title, ArtistId, INITCAP(Title) AS _cleaned_title,
CHAR_LENGTH(Title) AS _nb_char_in_title FROM bq_project_name.chinook.Album
““
```

et une vue sera générée dans le dataset “dbt_quad”.

La documentation et les tests dans DBT

Après avoir vu comment modéliser les données avec dbt, ce nouveau chapitre aborde les étapes de documentation et de tests dans DBT.

En effet, dbt permet de documenter les modèles de données pour : - améliorer la lisibilité, - faciliter la collaboration, - garder une trace claire de chaque transformation et de sa logique métier.

Ensuite, dbt inclut des fonctionnalités de test intégrées pour garantir la qualité et la fiabilité des données : - tests de non-nullité, - tests d’unicité, - tests de relations entre tables, etc.

Enfin, il est possible de personnaliser le comportement de chaque modèle via des fichiers de configuration comme la gestion de la matérialisation (table, vues, incrémentale), la priorisation des modèles et la gestion de rafraichissement.

La documentation dans DBT

Documenter les données est essentiel pour améliorer la communication entre équipes et garantir l’indépendance de celles-ci. Cela facilite aussi la maintenabilité du code. dbt permet de documenter les modèles via des fichiers *schema.yml*. Ces fichiers contiennent des descriptions des modèles et de leurs colonnes clés, en particulier celles spécifiques aux besoins métiers. dbt génère automatiquement un site web avec la commande `dbt docs generate` pour faciliter la visualisation et diffusion de cette documentation.

Une documentation bien faite répond aux questions récurrentes des équipes métiers, comme le calcul des KPIs, et évite des échanges répétitifs. Elle accélère également l'onboarding des nouvelles recrues dans une équipe data, car elles peuvent directement accéder aux informations nécessaires sur les modèles et données.

La documentation générée peut être visualisée localement via la commande `dbt docs serve`, ou directement dans dbt Cloud sans avoir besoin de la générer manuellement. Un des éléments clés de cette documentation est le lineage, une vue en arborescence qui montre les dépendances entre modèles, facilitant la compréhension des relations entre eux.

Il y a deux façons d'écrire de la documentation, la première est de rédiger un fichier `.yml` pour tous les modèles. Par exemple, dans `staging_mod_docs.yml`,

```
version: 2

models:
- name: stg_chin__albums      description: "This model contains information about albums"
- name: AlbumId              description: "Primary key, unique identifier for each album"
- unique
- not_null
- name: Title                description: "information about titles of albums"
- name: ArtistId             description: "Foreign key linking the album to the corresponding artist"
tests:
- relationships:              name: artist_id_foreign_key_in_stg_chin_album
  to: ref('stg_chin__artists') field: ArtistId
- name: _cleaned_title        description: "Name of the album with the first letter in upper case"
- name: _nb_char_in_title     description: "Count the number of character in the album title"

- name: stg_chin__artists     description: "This model contains information about artists"
- name: ArtistID              description: "Primary key, unique identifier for each artist."
tests:
- unique
- not_null
- name: Name                  description: "information about the name of the artists"
- name: _cleaned_name         description: "Make sure the name of the artist start with an upper case letter"
- name: _nb_char_name         description: "Count the number of character in the name of the artist"
- name: _snd_name             description: "Convert the name of the artist in sound characters"
```

on a la description de deux modèles dans le même fichier. Ce fichier `.yml` se structure de la manière suivante :

- `name: nom_du_modele` `description: description du modèle` `columns: ->`
Les colonnes du modèle
- `name: nom_de_la_colonne` `description: description de la colonne` ““

Une autre manière d'écrire de la documentation est d'écrire des blocs de code markdown qui seront ensuite référencés dans le fichier `.yml`. Ces blocs markdown

sont plus personnalisables (caractère en gras, italique ... puces, etc.) que la propriété “description” des fichiers yaml.

Par exemple, dans le dossier des modèles “intermediate” le fichier `int_chin__users_invs.md` :

```
{% docs int_chin__users_invs %}
```

This model provides a summary of user activity and invoices, focusing on their order history

Total Amount Spent: The total amount spent by the user.

Total Tracks: The total quantity of tracks purchased by the user.

Total Distinct Tracks: The count of distinct tracks purchased by the user.

Total Orders: The total number of orders placed by the user.

Dominant State : The state where most tracks are purchased Number of employee to manage

Average unit price : The average of unit price of tracks sold.

Total Distinct Customers : The count of distinct customer by tracks.

Total Support Solicitations : The number of times support is asked.

In top ten : True/False field to see if the track is in top ten of sales.

```
{% enddocs %} ```
```

Un ou plusieurs blocs markdown peuvent être écrits pour être référencé dans un fichier `.yaml`

version: 2

```
models: - name: int_chin__employees description: '{{ doc("int_chin__users_invs")
}}' columns: - name: CustomerId description: "Primary key, unique identifier
for each track." tests: - unique - not_null - name: _cust_full_name description:
"The full name of the customer" - name: Country description: "The country of
the manager" - name: total_support_solicitations description: "the total of
solicitations of the support" - name: _manager_full_name description: "The
full name of the manager" - name: nb_employees_to_manage description:
"The number of employees manage by the manager" ""
```

Au niveau de la propriété “description” du modèle, on trouve une référence à notre bloc markdown. Cette référence peut être réutilisée dans plusieurs fichiers YAML.

Les tests dans DBT

Le fait de tester les données permet de garantir leur qualité et d’éviter des erreurs courantes dans les dashboards d’entreprise (écarts de chiffres entre tableaux, erreurs de segmentation des données, etc.). En plus des tests basiques, des tests plus avancés comme la détection d’anomalies peuvent être réalisés pour intervenir avant que les données problématiques ne soient chargées en base.

Les tests dans dbt sont des affirmations sur les données. Par exemple : “chaque commande doit avoir un montant supérieur à zéro” ou “chaque utilisateur doit

avoir un `user__id` unique et non nul". Deux types de tests existent :

- Tests singular : Des requêtes SQL personnalisées, stockées dans le dossier `/test` du projet.
- Tests generic : Des tests standards (unicité, valeurs acceptées, non null, etc.) définis dans les fichiers YAML. Ces tests sont précodés dans dbt et ne nécessitent pas d'écrire de SQL.

Les tests s'exécutent avec la commande `dbt test`. Il est possible de lancer tous les tests ou de se limiter à un modèle spécifique avec `dbt test --select nom_modèle`. Si un test échoue, dbt renvoie une liste des erreurs pour les analyser et les corriger.

Les tests génériques

Les tests génériques sont à intégrer dans le fichier yaml contenant la documentation. Dans `staging_mod_docs.yml`,

```
version: 2
```

```
models:
```

```
- name: stg_chin__albums      description: "This model contains information about albums"
- name: AlbumId              description: "Primary key, unique identifier for each album"
- unique
- not_null
- name: Title                description: "information about titles of albums"
- name: ArtistId             description: "Foreign key linking the album to the corresponding artist"
tests:
```

```
- relationships:              name: artist_id_foreign_key_in_stg_chin_album
  to: ref('stg_chin__artists') field: ArtistId
- name: _cleaned_title        description: "Name of the album with the first letter in upper case"
- name: _nb_char_in_title     description: "Count the number of character in the album title"
```

on trouve l'implémentation d'un test d'unicité et d'un test non-null sur la colonne de clé p

- name: stg_chin__genre description: "This model contains details of products available for sale, primarily focusing on information about the product's volume and size." columns:
- name: GenreId description: "Primary key, unique identifier for each genre of music." tests:
- unique
- not_null
- name: Name description: "The label of the genre" tests:
- accepted_values: values: ['Rock', 'Science Fiction', 'Drama', 'Alternative & Punk', 'Pop', 'Metal', 'Latin', 'World', 'Soundtrack', 'Sci Fi & Fantasy', 'Blues', 'R&B/Soul', 'Rock And Roll', 'Electronica/Dance', 'TV Shows', 'Jazz', 'Heavy Metal', 'Opera', 'Bossa Nova', 'Classical', 'Alternative', 'Reggae', 'Easy Listening', 'Hip Hop/Rap', 'Comedy']

- name: `__cleaned__name` description: “The label of the genre with the first letter in upper case”
- name: `__nb_char__name` description: “The number of characters of the genre” “

Les tests singuliers

Les tests singuliers sont implémentés dans le dossier “tests” de notre projet dbt.

Par exemple, le test `quantity__positive.SQL` lance une requête SQL sur le modèle `stg__chin__invoice_lines` pour vérifier si les valeurs de la colonne “Quantity” ne sont pas négatives. Le test en lui-même repose sur une requête SQL qui cherche des valeurs négatives. S’il en trouve une, le test échoue.

```
select
InvoiceLineId, from  {{ ref('stg_chin__invoice_lines.sql') }} where Quantity < 0 ``
```

Travailler en collaboration avec Git

En entreprise, l'utilisation de Git comme système de gestion de versions est essentielle pour

Une des pratiques clés associées à Git est le "peer review" ou revue de code par les pairs. Ce processus favorise non seulement la qualité du code, mais aussi le partage des connaissances

De plus, Git offre des fonctionnalités comme les "branch" et le "merge", qui aident à gérer Ainsi, Git devient un outil indispensable pour maintenir un flux de travail efficace et coll

Rappel Git

Git est un outil puissant pour le contrôle de version, et maîtriser ses commandes essentielles

`*git add*` est utilisé pour ajouter des modifications ajoutées dans l'index (=staging) prépar

Avec `*git commit*`, vous pouvez enregistrer les modifications ajoutées dans l'index avec un m

`*git pull*` et `*git push*` sont essentiels pour la synchronisation avec un dépôt distant : `*git`

`*git branch*` est utilisé pour lister, créer ou supprimer des branches, ce qui est fondamenta

Enfin, `*git merge*` permet de fusionner les modifications d'une branche dans une autre, facilit

Une formation certifiante Git Kraken est disponible pour valider les commandes essentielles

Apporter ces modifications à la branche principale

Par convention, on ne travaille pas directement sur la branche "master" ou "main" d'un projet

`*git checkout -b nom-branch*` permet de créer et naviguer entre les branches d'un projet. Une

Workflow Git

1. Je mets à jour ma branche master/main

```
*git checkout master*
```

```
*git pull*
```

2. Je crée une branche

```
*git checkout -b ma-branche-avec-une-nouveaute*
```

3. Je fais mes modifications sur les fichiers du projet, j'ajoute mes modifications au dossier

```
*git add .*
```

```
*git commit -m «mon message de commit »*
```

4. J'envoie mes modifications sur le serveur Github/Gitlab

```
*git push origin ma-branch-avec-une-nouveaute*
```

5. Sur Github ou GitLab je crée une pull request : La PR déclenchera la CI/CD, qui doit être

6. Je demande une review à un collaborateur une fois la CI/CD passée. Si des changements sont

7. Une fois validée, je fusionne la PR avec la branche de production.

Git merge with main

La commande `*git merge master*` intègre les changements récents de master dans votre branche

Git rebase

La commande `*git rebase master*` applique vos commits après ceux de master, produisant un his

Gérer les conflits git

Lors de merges ou de rebase, des conflits peuvent apparaître si des modifications ont été ap

Les conflits peuvent être gérés directement dans un IDE comme VSCode ou dans l'interface Git

A. Cas pratique : Gérer un conflit lors d'un git merge

Dans un dépôt local :

1. Création d'une branche ajout-bonjour `*git checkout -b ajout-bonjour*`.

2. Création d'un fichier "conflict.txt" contenant sur la première ligne "Bonjour et bienvenue"
3. `*git add .*`, `*git commit "mon nouveau fichier bonjour"*`, `*git push*`

Sur Github :

4. Création et validation de la pull request et fusion de la branche ajout-bonjour à main

Dans le dépôt local :

5. Création d'une branche add-hello `*git checkout -b add-hello*`.
6. Création d'un fichier "conflict.txt" contenant sur la première ligne "Hello and welcome to the world"
7. `*git add .*`, `*git commit "my new file hello"*`, `*git push*`

Sur Github :

10. Résolution du conflit avec l'interface Github en cliquant sur "Resolve conflit" et une fois

Dans le dépôt local :

11. `*git checkout main*`
12. `*git pull*`

B. Cas pratique : Gérer un conflit lors d'un git rebase

Dans un dépôt local :

1. Création d'une branche ajout-bonjour `*git checkout -b ajout-aurevoir*`.
2. Création d'un fichier "conflict2.txt" contenant sur la première ligne "Merci d'avoir suivi ce tutoriel"
3. `*git add .*`, `*git commit "mon nouveau fichier au revoir"*`, `*git push*`
4. Création et validation de la pull request et fusion de la branche ajout-aurevoir à main
5. Création d'une branche add-goodbye `*git checkout -b add-goodbye*`.
6. Création d'un fichier "conflict2.txt" contenant sur la première ligne "Thanks for following me"
7. `*git add .*`, `*git commit "my new file goodbye"*`, `*git push*`
8. Récupération des modifications récemment fusionnées sur main sur la branche "add-goodbye"
9. Application des modifications de "main" sur la branche "add-goodbye" avec `*git rebase origin main*`
10. Résolution du conflit avec un éditeur de texte.
11. Ajout des modifications à l'index `*git add conflict2.txt*`
12. `*git rebase continue*` et `*git push origin add-goodbye --force*`

Sur github :

13. Création et validation de la pull request et fusion de la branche ajout-goodbye à main

Mettre son code en production

Les pull requests : Bonnes pratiques

Une Pull Request (PR) regroupe les modifications apportées et propose leur intégration sur la branche main. Elles doivent être créées sur le dépôt distant à la suite de la commande `*git push branche-cible*`

Elle doit être accompagnée d'une description claire pour expliquer les changements. En entre

Concernant les branches, utilisez des lettres minuscules, des tirets pour séparer les mots, Dans les commit, précisez l'action avec des verbes et ajoutez un préfixe pour clarifier l'in Enfin, comme pour les branches et commits, le titre de la PR doit être explicite, et la des

Paramétrer son dépôt distant au code review

Sur Github dans notre dépôt distant :

1. Se rendre dans l'onglet "Settings"
2. Se rendre dans les options "Branches"
3. Cliquer sur "Add classic branch protection rule"
4. Désigner la branche sur laquelle on souhaite appliquer les règles, en l'occurrence "main"
5. Cocher les règles suivantes :
 - Require a pull request before merging : empêche la modification de la branche main directe
 - Require approval : Nécessite une approbation d'un tiers
 - Require review from code owners
 - Require conversation resolution before merging
 - Lock branch
5. Cliquer sur "Create"

Mettre en place une CI/CD

Définition

Le CI/CD (Continuous Integration/Continuous Deployment) est un ensemble de workflows automat

Au cours de sa mission l'analytics engineer peut rencontrer ces erreurs :

- Code SQL mal formaté : les entreprises utilisent souvent des linters comme SQLFluff. Si le
- Code dbt non compilable : le code dbt peut parfois ne pas compiler correctement ; la CI/CD
- Échec de tests dbt : si des tests dbt sont intégrés dans le CI/CD, tout test échoué sera s
- Gestion incorrecte des dépendances : un renommage de colonne peut provoquer des erreurs da

La CI/CD prévient les erreurs en production grâce à des vérifications automatisées et permet

Github actions

GitHub Actions est un outil d'intégration continue et de livraison continue (CI/CD) qui perm

Pour mettre en place GitHub Actions dans un projet, commencez par créer un répertoire nommé

Voici un exemple simple de fichier YAML pour un workflow qui exécute des tests à chaque push

name: CI

on: [push]

```
jobs: jobname: runs-on: ubuntu-latest steps: - uses: actions/checkout@v2 -
name: Run a one-line script run: echo Hello, world! - name: Run tests run: npm
test ""
```

Dans cet exemple, le workflow est déclenché à chaque push vers le dépôt. Il utilise un container avec la dernière version d'Ubuntu, effectue un checkout du code, exécute une commande simple pour afficher "Hello, world!", et enfin, exécute les tests avec npm. Une fois votre fichier YAML créé et poussé vers votre dépôt, GitHub Actions exécutera automatiquement le workflow selon les événements spécifiés.

Pour contrôler et tester notre code, on écrira ce workflow :

```
name: CICD DBT
# https://github.com/marketplace/actions/dbt-action
# https://docs.github.com/en/actions/writing-workflows/quickstart run-name: ${ github.actor }

# Run jobs when a pull request is created on: [pull_request]

jobs:  action:
# Create an Ubuntu container      runs-on: ubuntu-latest

steps:
- run: echo " The job was automatically triggered by a ${ github.event_name } event."
- run: echo " This job is now running on a ${ runner.os } server hosted by GitHub!"
- run: echo " The name of your branch is ${ github.ref } and your repository is ${ github.repository }"
# Clone the repos
- name: Checkout repository      uses: actions/checkout@v4
- run: echo " The ${ github.repository } repository has been cloned to the runner."
- run: echo " The workflow is now ready to test your code on the runner."
# Run dbt
- name: dbt-run      uses: mwhitaker/dbt-action@master      with:
# Get latest dependancies      dbt_command: "dbt deps"
# Run dbt on the latest changes with our profile      dbt_command: "dbt run - select +s
dbt_project_folder: "uv_dag_dbt_bq"      env:
# BigQuery credentials in secrets of our github project
DBT_BIGQUERY_TOKEN: ${ secrets.DBT_BIGQUERY_TOKEN }
- name: List files in the repository      run: |
ls ${ github.workspace }
- run: echo " This job's status is ${ job.status }."
```

Conclusion

En conclusion, cette formation d'Analytics Engineer nous a permis d'acquérir des compétences essentielles pour transformer des données brutes en informations exploitables grâce à des outils puissants tels que dbt, BigQuery et Git.

Nous avons appris à structurer et à modéliser des données de manière efficace avec dbt, à exploiter la puissance de BigQuery pour effectuer des analyses à grande échelle, et à collaborer de manière optimale grâce à Git.

Ces compétences nous positionnent favorablement pour relever les défis du monde de la data et contribuer de manière significative à la prise de décision stratégique au sein des organisations. Nous sommes désormais équipés pour concevoir des pipelines de données robustes, assurer leur maintenance et favoriser une culture de la donnée au sein des équipes.